Introduction to Machine Learning

Machine Learning (ML) is a rapidly growing field within artificial intelligence (AI) that focuses on building systems capable of learning from data. Instead of being explicitly programmed with detailed rules, the ML models identify patterns and make predictions or decisions based on historical data. This approach has revolutionized many industries, including healthcare, finance, marketing, and technology, enabling applications like personalized recommendations, fraud detection, and speech recognition. As the volume of data continues to grow, understanding ML concepts and techniques becomes increasingly important for anyone interested in working with data or building intelligent systems.

This course provides a comprehensive introduction to the fundamental principles and techniques of ML. We will cover essential topics such as supervised learning, unsupervised learning, and model evaluation, as well as the basics of data preprocessing and feature selection. The participants will learn how to design and implement basic ML models using tools and frameworks commonly used in the industry. Through a combination of theory, practical exercises, and real-world examples, the participants will gain a solid foundation in ML, preparing them for further study or practical application in various domains.

By the end of the course, the participants will understand how the ML models are built, how to select the appropriate algorithms for specific problems, and how to assess model performance. In addition, the participants will be equipped with the skills to work on ML projects, from data collection and preparation to model training and evaluation. This knowledge will serve as a valuable stepping stone for those wishing to explore more advanced topics or specialize in areas such as deep learning (DL), natural language processing, or computer vision.

⚙ Prerequisites

- Familiarity with Python basics (loops, functions, lists, dictionaries) and libraries like NumPy, Pandas, and Matplotlib/Seaborn
- Understanding vectors, matrices, matrix operations (multiplication, transpose, inverse), and eigenvalues
- (Optional but useful): Derivatives, gradients, and optimization (especially for understanding gradient descent)

20 min	filename

Introduction to Machine Learning

Objectives

Questions

- Describe a general description of ML
- Clarify the relationship between AI, ML, and DL
- Get familiar with representative real-word applications of ML

- What is Machine Learning?
- What is the relationship between AI, ML, and DL?
- Why should we embrance ML at the current stage?

❗ Objectives

- Describe a general description of ML
- Clarify the relationship between AI, ML, and DL
- Get familiar with representative real-word applications of ML

❓ Questions

- What is Machine Learning?
- What is the relationship between AI, ML, and DL?
- Why should we embrance ML at the current stage?

What is Machine Learning

Machine learning (ML) is a field of computer science that studies algorithms and techniques for automating solutions to complex problems that are hard to program using conventional programing methods.

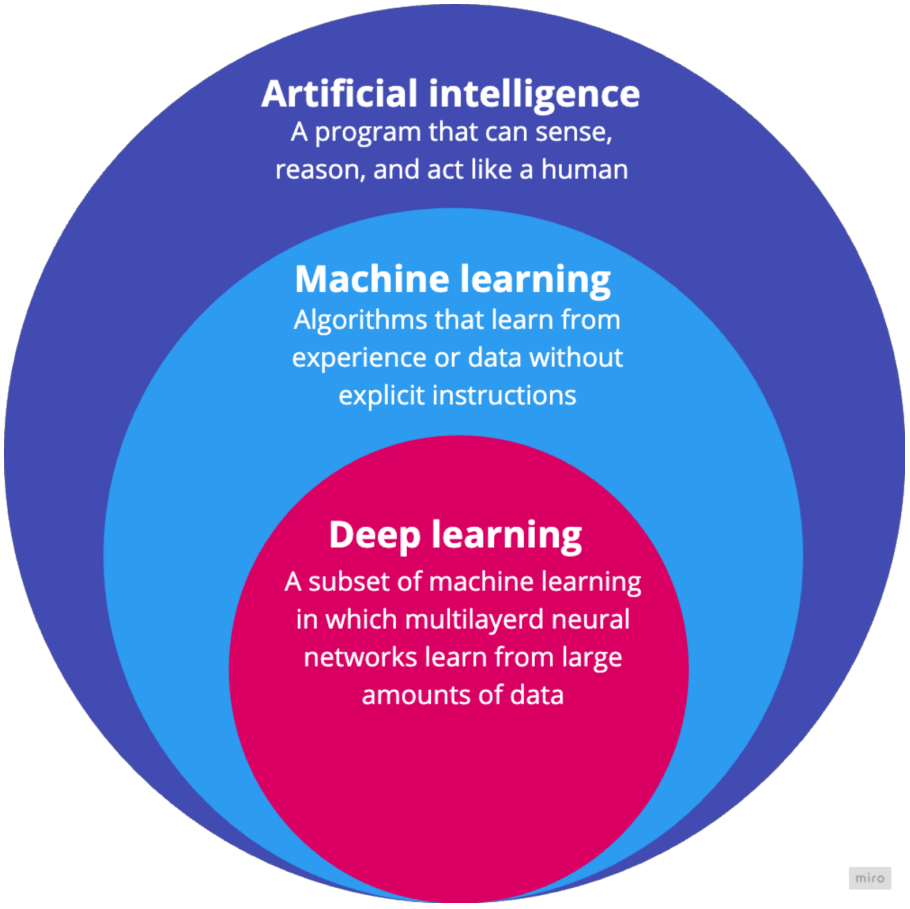
In conventional programming, the programmer explicitly codes the logic (rules) to transform inputs (data) into outputs (answers), making it suitable for well-defined, rule-based tasks. In ML, the system learns the logic (rules) from data and answers, making it ideal for complex, pattern-based tasks where explicit rules are hard to define. The choice between them depends on the problem, data availability, and complexity.

Relation with Artificial Intelligence and Deep Learning

Artificial Intelligence (AI) is the broadest field, encompassing any technique that enables computers to mimic human intelligence, such as reasoning, problem-solving, perception, and decision-making. AI includes a wide range of approaches, from rule-based systems (like expert systems) to modern data-driven methods. It aims to create systems that can perform tasks that typically require human intelligence, such as playing chess, recognizing images, or understanding language.

ML is a subset of AI that focuses on algorithms and models that learn patterns from data to make predictions or decisions without being explicitly programmed. ML is one of the primary ways to achieve AI. It enables systems to improve performance over time by learning from experience (data) rather than relying solely on hardcoded rules. ML includes various techniques like supervised learning (*e.g.*, regression, classification), unsupervised learning (*e.g.*, clustering, dimensionality reduction), and reinforcement learning.

Deep Learning (DL) is a specialized subset of ML that uses neural networks with many layers (hence “deep”) to model complex patterns in large datasets. DL is a subset of ML, and it leverages artificial neural networks inspired by the human brain to tackle tasks like image recognition, speech processing, and natural language understanding. DL excels in handling unstructured data (*e.g.*, images, audio, text) and requires significant computational power and large datasets for training.



The relationship between artificial intelligence, machine learning, and deep learning.

Why Machine Learning?

ML is transforming how we solve complex problems in the real world by enabling systems to learn directly from data, rather than relying on explicitly programmed rules. In many real-world scenarios, such as medical diagnosis, stock market prediction, or natural language processing, the relationships between inputs and outputs are too complex or dynamic to define manually. ML models can uncover hidden patterns and make accurate predictions or decisions, making them essential tools in fields like healthcare, finance, transportation, and cybersecurity.

Another crucial advantage of ML is its ability to adapt and improve over time as more data becomes available. Unlike traditional rule-based systems that require constant manual updates, ML models can retrain and adjust themselves to new data, trends, or anomalies, ensuring that the system stays relevant and effective. For example, in fraud detection, ML algorithms can evolve as fraud tactics change, providing a stronger defense compared to static rules that may become outdated. This adaptability makes ML particularly powerful in dynamic, real-time environments where traditional programming methods fall short.

In addition, ML empowers the automation of complex tasks that were previously dependent on human expertise and intuition. From voice recognition in virtual assistants to autonomous driving, ML algorithms can process vast amounts of unstructured data such as text, images, and audio, which are traditionally challenging for computers to handle. By enabling machines to "learn" from experience and improve their performance over time, ML not only enhances productivity but also opens new frontiers for innovation across industries, creating smarter systems that can make meaningful contributions to society.

ML is used across a wide range of industries and real-world problems in healthcare, finance, natural language processing, computer vision, transportation, manufacturing industry, retail, and cybersecurity.

Machine Learning Applications

Below are key categories of problems that can be applied using ML.

Problems can be solve with ML

ML is used across a wide range of industries and real-world problems in healthcare, finance, natural language processing, computer vision, transportation, manufacturing industry, retail, and cybersecurity.

Machine Learning Applications

Below are key categories of problems that can be applied using ML.

Problems can be solve with ML

Application area	Example use Cases
Healthcare	Disease prediction & diagnosis, medical image analysis, drug discovery
Finance	Fraud detection, credit scoring, algorithmic trading
Retail & e-commerce	Product recommendations, customer segmentation, demand forecasting
Transportation & autonomous systems	Self-driving cars, traffic prediction, route optimization
Natural language processing (NLP)	Chatbots and virtual assistants, sentiment analysis, language translation
Manufacturing & industry	Predictive maintenance, quality control, supply chain optimization
Computer Vision	Facial recognition, object detection, image classification

Problems can't be solve with ML

ML is powerful, but it's not magic. It's a tool for finding patterns in data but has no idea what the patterns mean. Therefore it is not a substitute for human reasoning, creativity, or ethical judgment.

Below are key categories of problems that cannot be solved with ML due to inherent limitations, regardless of data or computational advancements.

- Problems with insufficient or poor-quality data: ML relies heavily on data. If data is scarce, noisy, biased, or unrepresentative, models fail to generalize. For example, predicting rare events with limited historical data (*e.g.*, catastrophic asteroid impacts, spread of pandemic) is unreliable.
- Problems requiring reasoning, understanding, or deep logic. ML models approximate patterns but don't understand them. They lack reasoning and common sense unless explicitly designed (*e.g.*, symbolic AI).
- Problems that involve subjective judgments or value-based decisions. ML models don't "know" what's right or wrong – they reflect patterns in the data, including biases.
- Problems outside of distribution generalization. A model trained on photos of cats can't accurately classify dogs if it never saw dogs. ML models interpolate between known data. They struggle with novel scenarios far outside the training set.

Problems can be, but shouldn't be solved with ML

There are many problems where ML (or DL) could technically be applied, but shouldn't be – either because of high stakes, lack of transparency, or due to ethical, practical, or societal concerns.

- Tasks for modelling well defined systems, where the equations governing them are known and understood.
- Problems of high stakes, lack of transparency, or due to ethical, practical, or societal concerns.
  - ML can predict outcomes in fields like medical diagnosis or aviation safety, but even small errors can lead to catastrophic consequences. Over-reliance on ML without human oversight risks lives when models fail in edge cases.

- Tasks for modelling well defined systems, where the equations governing them are known and understood.

There are many problems where ML (or DL) could technically be applied, but shouldn't be – either because of high stakes domains with unacceptable ethical, practical, or societal concerns.

- Problems of high stakes domains with unacceptable ethical, practical, or societal concerns: ML can predict outcomes in fields like medical diagnosis or aviation safety, but even small errors can lead to catastrophic consequences. Over-reliance on ML without human oversight risks lives when models fail in edge cases.
- Privacy-sensitive applications: ML can analyze personal data (*e.g.*, health records, browsing habits) to predict behaviors, but using it for invasive profiling, surveillance, or targeted manipulation (*e.g.*, hyper-personalized propaganda) raises serious privacy and autonomy concerns.
- Reinforcing harmful social norms: ML can optimize systems like targeted advertising or content recommendation, but doing so can amplify harmful behaviors (*e.g.*, echo chambers, misinformation, or addiction to social media) if not carefully regulated.

Fundamentals of Machine Learning

❓ Questions

- what are the main types of MLs?

❗ Objectives

- Describe main types of ML

Types of Machine Learning

ML can be broadly categorized into three main types depending on how the models learn from input data and the nature of the input data they process.

Supervised learning

In supervised learning, the model is trained on a labeled dataset, where each input is paired with a corresponding output (label). The goal is to learn a mapping from inputs to outputs to make predictions on new, unseen data.

Supervised learning has two subtypes: **Classification** (predicting discrete categories) and **Regression** (predicting continuous values).

Here are representative examples of these two subtypes in real-word problems:

- **Classification**: email spam detection (spam/ham), image recognition (cat/dog), medical diagnosis (disease/no disease).
- **Regression**: house price prediction, weather forecasting.

Unsupervised learning

Unsupervised learning also has two subtypes: **Clustering** (grouping similar data points together) and **Dimensionality reduction** (simplifying data by reducing features while preserving important information)

In unsupervised learning, the model works with unlabeled data, identifying patterns, structures, or relationships within the data without explicit guidance on what to predict.

Representative examples of these two subtypes in real-word problems:

Unsupervised learning also has two subtypes: **Clustering** (grouping similar data points together) and **Dimensionality reduction** (simplifying data by reducing features while preserving important information)

In unsupervised learning, the model works with unlabeled data, identifying patterns, structures, or relationships within the data without explicit guidance on what to predict.

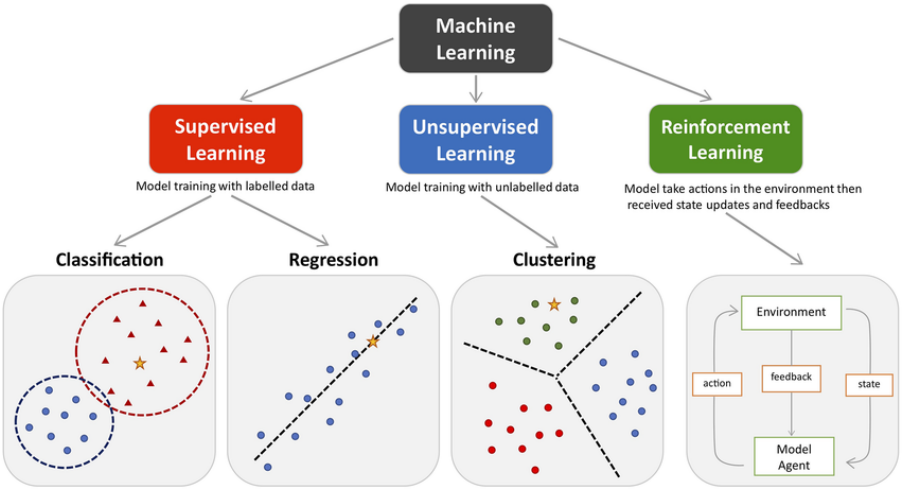
Representative examples of these two subtypes in real-word problems:

- **Clustering**: customer segmentation in marketing (grouping users by behavior), image segmentation (grouping similar pixels).
- **Dimensionality reduction**: compressing high-dimensional data (*e.g.*, reducing image features for faster processing), anomaly detection.

Reinforcement learning

The model (agent) learns by interacting with an environment. It takes actions, receives feedback (rewards or penalties), and learns a strategy (policy) to maximize long-term rewards.

Representative examples of reinforcement learning in real-word problems: game-playing AI (*e.g.*, AlphaGo), robot navigation, autonomous driving.



Three main types of machine learning. Main approaches include classification and regression under the supervised learning and clustering under the unsupervised learning. Reinforcement learning enhance the model performance by interacting with environment. Coloured dots and triangles represent the training data. Yellow stars represent the new data which can be predicted by the trained model. This figure was taken from the paper Machine Learning Techniques for Personalised Medicine Approaches in Immune-Mediated Chronic Inflammatory Diseases: Applications and Challenges.

Other subtypes

In addition to supervised and unsupervised learning, there are other important paradigms in machine learning.

- **Semi-supervised learning** bridges the gap between supervised and unsupervised learning by using a small amount of labeled data together with a large amount of unlabeled data, helping models learn more effectively when labeling is expensive or time-consuming (*e.g.*, medical image analysis).
- **Transfer learning** involves applying knowledge from a pretrained model, trained on a large, general dataset, to a new, related task, significantly reducing training time and data requirements (*e.g.*, fine-tuning a speech recognition model for a new dialect).
- **Self-supervised learning** is a form of unsupervised learning where the model generates its own labels from the data — typically for pretraining models on tasks like image or language understanding, enabling them to learn robust representations without explicit labels (*e.g.*, predicting the next word in a sentence, and filling in missing image patches)

These techniques expand the capabilities and versatility of machine learning across data-limited or computationally constrained environments.

- **Transfer learning** involves applying knowledge from a pretrained model, trained on a large, general dataset, to a new, related task, significantly reducing training time and data requirements (*e.g.*, fine-tuning a speech recognition model for a new dialect).
- **Self-supervised learning** is a form of unsupervised learning where the model generates its own labels from the data — typically for pretraining models on tasks like image or language understanding, enabling them to learn robust representations without explicit labels (*e.g.*, predicting the next word in a sentence, and filling in missing image patches)

These techniques expand the capabilities and versatility of machine learning across data-limited or computationally constrained environments.

Machine Learning Workflow

What is a workflow for ML?

A machine learning workflow is a structured approach for developing, training, evaluating, and deploying machine learning models. It typically involves several key phases, including data collection, preprocessing, model training and evaluation, and finally, deployment to production.

Here is a graphical representation of ML workflow, and a concise overview of the key steps are described below.

Problem definition and project setup

Problem Definition is the first and most critical phase of any ML project. It sets the direction, scope, and goals for the entire project.

- we should understand the problem domain: what is the real-world problem we are trying to solve? are we predicting, classifying, or grouping data? (*e.g.*, predict house prices, detect spam emails, cluster customers)
- we should determine if ML is the appropriate solution for the problem
- we then should identify the expected outputs: what will the ML model produce? (*e.g.*, a number, a label, or a probability)
- we define the type of ML task (*e.g.*, classification and regression tasks for supervised learning, clustering, dimensionality reduction for unsupervised learning, and decision-making tasks for reinforcement learning)

Project Setup is to set up the programming/development environment for the project.

- hardware requirements (CPU, SSD, GPU, cloud platforms, *etc.*)
- software requirements (programming languages and libraries, ML/DL frameworks, and development tools, IDEs, Git/Docker, *etc.)
- project structure: organize your project for clarity and scalability

A typical ML project structure looks like this

```
ML_Project/
├── data/                 # raw and processed data
│   ├── raw/              # original, unprocessed data
│   ├── processed/        # cleaned, preprocessed data
├── notebooks/            # jupyter notebooks for EDA & modeling
├── src/                  # source code
│   ├── utils/            # utility functions (*e.g.*, metrics, logging)
│   ├── preprocessing.py  # data cleaning script
```

```
ML_Project/
├── data/                 # raw and processed data
│   ├── raw/              # original, unprocessed data
│   ├── processed/        # cleaned, preprocessed data
├── notebooks/            # jupyter notebooks for EDA & modeling
├── src/                  # source code
│   ├── utils/            # utility functions (*e.g.*, metrics, logging)
│   ├── preprocessing.py  # data cleaning script
│   └── train.py          # model training script
├── models/               # trained model files (*e.g.*, .pkl, .h5)
├── tests/                # unit and integration tests
├── README.md             # project overview and setup instructions
├── requirements.txt      # project dependencies
├── config.yaml           # configuration file for hyperparameters and paths
```

Data collection and preprocessing

In ML, data collection and preprocessing are crucial steps that significantly affect the performance of a model. High-quality, well-processed data leads to better predictions, while poor data can result in unreliable models.

- **Data collection**: Gather the necessary data from various sources (*e.g.*, databases, APIs (twitter, linkedin, *etc.*), or manual collection), and ensure that data is representative and sufficient for the problem.
- **Data preprocessing**: Clean and prepare data by handling missing values (drop, impute, or predict), removing duplicates or irrelevant data, fixing inconsistencies (*e.g.*, “USA” vs. “United States”), normalizing/scaling features, encoding categorical variables, and addressing outliers, and other data quality issues.
- **Exploratory data analysis (EDA)**: Analyze data to uncover distributions, correlations, patterns, anomalies, and insights using visualizations and statistical methods. This helps in feature selection and understanding data distribution.
- **Feature engineering**: Create or select relevant features to improve model performance. This may involve dimensionality reduction (*e.g.*, PCA (principal component analysis)) or creating new features based on domain knowledge.
- **Data splitting**: Divide the dataset into training, validation, and test sets (*e.g.*, 70-15-15 split) to evaluate model performance and prevent overfitting.

Model selection and training

Model Selection and Training refer to the process of choosing an appropriate model architecture and training it to learn patterns from data to solve a specific task. It involves selecting the appropriate algorithms (*e.g.*, linear/logistic regression, decision trees, neural networks, Gradient Boosting) based on the problem type, configuring its hyperparameters, and optimizing its parameters using training data to minimize error or maximize performance metrics.

Model evaluation and assessment

Model evaluation and assessment in machine learning refers to the process of measuring and analyzing a model’s performance to determine its effectiveness in solving a specific task. It involves using metrics and techniques to quantify how well the model generalizes to unseen data, identifies patterns, and meets desired objectives, typically using a test dataset separate from the training data.

Below are common evaluation metrics by task types:

Task types	Evaluation metrics
Classification	Accuracy, precision, recall, F1-score, ROC-AUC, *etc.*
Regression	Mean Squared Error (MSE), Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), R-squared, *etc.*

Model evaluation and assessment in machine learning refers to the process of measuring and analyzing a model's performance to determine its effectiveness in solving a specific task. It involves using metrics and techniques to quantify how well the model generalizes to unseen data, identifies patterns, and meets desired objectives, typically using a test dataset separate from the training data.

Below are common evaluation metrics by task types:

Task types	Evaluation metrics
Classification	Accuracy, precision, recall, F1-score, ROC-AUC, *etc.*
Regression	Mean Squared Error (MSE), Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), R-squared, *etc.*
Clustering	Silhouette score, Davies-Bouldin index, Calinski-Harabasz index.
Ranking	Mean Reciprocal Rank (MRR), Normalized Discounted Cumulative Gain (NDCG).
NLP or generative tasks	BLEU, ROUGE, perplexity (often overlaps with deep learning).

Here are representative techniques and processes for the assessment:

- **Train-validation-test split**: Divide data into training (model learning), validation (hyperparameter tuning), and test (final evaluation) sets to prevent overfitting.
- **Cross-validation**: Use k-fold cross-validation to assess model stability across multiple data subsets.
- **Confusion matrix**: For classification, visualize true positives, false negatives, etc.
- **Learning curves**: Plot training *vs.* validation performance to diagnose underfitting or overfitting.
- **Comparison with baselines**: Comparing model performance against simple baselines (*e.g.*, random guessing, linear models) to ensure meaningful improvement.
- **Robustness testing**: Evaluate performance under noisy, adversarial, or out-of-distribution data.
- **Fairness and bias analysis**: Assess model predictions for fairness across groups (*e.g.*, demographics).

Hyperparameter tuning

Hyperparameter tuning is the process of optimizing the settings (hyperparameters) of a model that are not learned during training but significantly affect its performance. These include parameters like learning rate, number of hidden layers, or batch size, which control the model's behavior and training process.

The goal of this process is to find the best combination of hyperparameters that maximizes performance metrics (*e.g.*, accuracy, precision) on a validation set.

Model deployment, monitoring, and improvement

Model deployment, monitoring, and improvement refer to the processes involved in taking a trained machine learning model from development to production, ensuring it performs effectively in real-world applications, and continuously enhancing its performance.

- **Model deployment** indicates an integration of a trained model into a production environment (APIs or cloud platforms) where it can make predictions or decisions on new, unseen data.
- As the models degrade over time, so continuous improvement is necessary. **Model improvement** involves updating or retraining the model to maintain or enhance its performance based on monitoring insights or new data.
- Once deployed, the model's performance must be continuously tracked to ensure it remains accurate and reliable over time, which is termed as **model monitoring**.

Machine Learning Libraries

Scikit-learn

- **Model deployment** indicates an integration of a trained model into a production environment (APIs or cloud platforms) where it can make predictions or decisions on new, unseen data.
- As the models degrade over time, so continuous improvement is necessary. **Model improvement** involves updating or retraining the model to maintain or enhance its performance based on monitoring insights or new data.
- Once deployed, the model's performance must be continuously tracked to ensure it remains accurate and reliable over time, which is termed as **model monitoring**.

Machine Learning Libraries

Scikit-learn

Scikit-learn is a widely-used, open-source Python library designed for **classical machine learning**, offering a variety of algorithms and tools for for tasks, such classification, regression, clustering, and dimensionality reduction. It supports supervised learning (*e.g.*, SVM (support vector machine), decision trees, random forests), unsupervised learning (*e.g.*, k-means, PCA (principal component analysis)), and semi-supervised learning, with robust tools for data preprocessing, model evaluation, and hyperparameter tuning via `GridSearchCV`. Built on NumPy, SciPy, and Matplotlib, it is designed for ease of use, making it ideal for beginners and rapid prototyping. Scikit-Learn excels in handling small to medium-sized datasets and includes utilities for data preprocessing, model evaluation, hyperparameter tuning, and pipeline construction. However, it lacks support for DL and GPU acceleration, limiting its scalability for large datasets or complex neural network tasks.

Keras

Keras is a high-level neural networks API that simplifies the process of building and training DL models. Originally an independent library, Keras is now tightly integrated with TensorFlow as its official high-level interface (but also usable standalone), offering an accessible way to experiment with DL without sacrificing performance. Keras provides user-friendly abstractions for layers, models, loss functions, and optimizers, allowing users for quick prototyping of neural networks for tasks like image classification, text generation, and time series forecasting with minimal code. Keras abstracts away much of the complexity of TensorFlow while retaining flexibility, making it ideal for beginners and those who need fast experimentation.

TensorFlow

Developed by Google, **TensorFlow** is a powerful open-source library primarily for DL but versatile enough for a broad range of ML tasks. It provides a flexible ecosystem for building complex models, including neural networks for computer vision, natural language processing, and time series analysis. TensorFlow supports distributed computing across CPUs, GPUs, and TPUs, making it suitable for both research and production at scale. Its robust features, such as TensorBoard for visualization, TensorFlow Serving for model deployment, and TensorFlow Lite for mobile inference, make it a comprehensive framework for end-to-end machine learning development. TensorFlow's high-level Keras API simplifies model building, while its low-level operations provide flexibility for advanced research. TensorFlow is well-suited for tasks like image recognition, natural language processing (NLP), and reinforcement learning, though its complexity can pose a steeper learning curve for beginners compared to alternatives like PyTorch.

PyTorch

Developed by Facebook's AI Research Lab (FAIR), PyTorch is a user-friendly and open-source DL library that has gained significant popularity in academia and industry. Known for its intuitive design and "define-by-run" (eager execution) approach, PyTorch allows developers to build, train, and debug models in a flexible and interactive manner. Its strong support for GPU acceleration and extensive ecosystem-ranging from computer vision (TorchVision) to NLP (TorchText) and audio (TorchAudio) – make it an excellent choice for cutting-edge DL research and production. Popular in academia and increasingly in industry, PyTorch excels in rapid prototyping and experimentation but is less optimized for production deployment compared to TensorFlow. Its active community and support for GPU acceleration make it a

PyTorch

Developed by Facebook's AI Research Lab (FAIR), PyTorch is a user-friendly and open-source DL library that has gained significant popularity in academia and industry. Known for its intuitive design and "define-by-run" (eager execution) approach, PyTorch allows developers to build, train, and debug models in a flexible and interactive manner. Its strong support for GPU acceleration and extensive ecosystem-ranging from computer vision (TorchVision) to NLP (TorchText) and audio (TorchAudio) – make it an excellent choice for cutting-edge DL research and production. Popular in academia and increasingly in industry, PyTorch excels in rapid prototyping and experimentation but is less optimized for production deployment compared to TensorFlow. Its active community and support for GPU acceleration make it a favorite for cutting-edge ML and DL research.

XGBoost & LightGBM

XGBoost (Extreme Gradient Boosting) and **LightGBM** (Light Gradient Boosting Machine) are high-performance gradient boosting libraries that have become go-to solutions for structured data problems, such as tabular datasets. Both libraries implement optimized gradient boosting algorithms that deliver fast training speeds, high accuracy, and scalability to large datasets. XGBoost is known for its robustness and versatility, while LightGBM offers further speed and memory efficiency through histogram-based algorithms and leaf-wise growth strategies. These libraries have become essential tools for data scientists working with structured data, outperforming traditional models in many real-world scenarios.

Hugging Face Transformers

Hugging Face Transformers is a cutting-edge library that provides access to state-of-the-art pre-trained models for NLP tasks and computer vision, including text classification, translation, summarization, and question answering. The library's pre-trained models and tokenizers simplify NLP workflows by enabling rapid experimentation with large language models, and in addition, this library supports both TensorFlow and PyTorch backends, integrating with datasets via Hugging Face's datasets library, and has a vibrant community contributing to its continuous development.

FastAI

FastAI is a high-level DL library built on PyTorch, designed to make AI accessible to a wider audience by simplifying complex tasks. It provides high-level abstractions and best practices out-of-the-box, allowing users to train powerful models with minimal code and optimal defaults. FastAI is particularly well-known for its transfer learning capabilities, enabling quick adaptation of pre-trained models for tasks like image classification and text generation. With its focus on practical usage, education, and strong community support, FastAI is ideal for beginners and practitioners who want to quickly deploy models without deep theoretical expertise.

JAX

JAX, developed by Google, combines NumPy-like syntax with automatic differentiation and GPU/TPU acceleration, making it ideal for high-performance ML research. It enables composable function transformations (gradients, JIT compilation) and scales efficiently across hardware. While not as high-level as TensorFlow or PyTorch, JAX is favored for cutting-edge numerical computing, physics simulations, and advanced neural network research where speed and flexibility are crucial.

These libraries cater to different needs: Scikit-learn for classical ML, TensorFlow and PyTorch for DL and scalability, Keras for simplicity, XGBoost for high-performance tabular data tasks, and Hugging Face for transformer-based applications. The choice of these libraries depends on the task, data type, scalability needs, user expertise, and whether the focus is research, prototyping, or production deployment.

A summary of best features and key strengths of these libraries are summarized below.

Library	Best Feature	Key Strength

cutting-edge numerical computing, physics simulations, and advanced neural network research where speed and flexibility are crucial.

These libraries cater to different needs: Scikit-learn for classical ML, TensorFlow and PyTorch for DL and scalability, Keras for simplicity, XGBoost for high-performance tabular data tasks, and Hugging Face for transformer-based applications. The choice of these libraries depends on the task, data type, scalability needs, user expertise, and whether the focus is research, prototyping, or production deployment.

A summary of best features and key strengths of these libraries are summarized below.

Library	Best Feature	Key Strength
Scikit-Learn	Simple and consistent API for classical ML tasks (classification, regression, clustering) and small/medium datasets.	Seamless integration with NumPy/Pandas and extensive documentation for ease-of-use with wide algorithm support
PyTorch	Dynamic computation graph (define-by-run) for flexible model building and debugging	Flexible, intuitive framework with strong adoption for academic research in DL tasks
TensorFlow	Scalability with GPU/TPU acceleration for complex deep learning models	Excellent ecosystem (Keras, TF Hub, TF-Agents) for production-scale applications
Keras	High-level, user-friendly API for rapid prototyping	Simplifies construction of DL models, making it beginner-friendly and efficient with TensorFlow compatibility for quick model development
XGBoost & LightGBM	Optimized gradient boosting algorithms	Extremely effective for high-performance supervised learning with tabular/structured data
Hugging Face Transformers	Extensive pretrained transformer models for easy fine-tuning	Community-driven ecosystem with user-friendly pipelines for NLP and vision tasks
FastAI	Transfer learning made easy for NLP & vision tasks	Fast prototyping with minimal code and strong performance for applied deep learning
JAX	NumPy + autodiff + GPU/TPU acceleration	Cutting-edge numerical computing, works with PyTorch/TensorFlow via interoperability libraries, but offers lower-level control

Scientific Data for Machine Learning

❓ Questions

- why data is super important in ML?
- what is the type and form of scientific data?
- what are the representative data storage formats?
- which data structure is widely used in ML/DL applications?
- what is data processing and feature engineering in ML/DL applications?

Data is the Foundation of Machine Learning

Data is the backbone of ML as it serves as the foundation for training models to recognize patterns, make predictions, and generate insights. Without high-quality, relevant, and well-structured data, ML algorithms cannot learn meaningful patterns or make accurate predictions. Poor or insufficient data leads to biased, unreliable, or ineffective AI systems. Moreover, data preparation and processing consume a significant portion of the ML workflow, often more time than model development itself. Cleaning, transforming, and structuring raw data into a usable format ensures that algorithms can extract valuable insights efficiently. The choice of data formats, like CSV for simplicity or HDF5 for large-scale datasets, also impacts data storage, accessibility, and computational efficiency during model training and deployment.

From image recognition to natural language processing, every ML application depends on properly curated datasets for training, validation, and testing. In addition, data determines the applicability and scalability of ML solutions across domains, from scientific research to real-world applications.

Moreover, data preparation and processing consume a significant portion of the ML workflow, often more time than model development itself. Changing transformations and structured data, ML algorithms cannot learn meaningful patterns or make accurate predictions. Poor or insufficient data leads to biased, unreliable, ineffective AI systems. From image recognition to natural language processing, every ML application depends on properly curated datasets for training, validation, and testing. In addition, data determines the applicability and scalability of ML solutions across domains, from scientific research to real-world applications.

In this episode , we will

Understanding Scientific Data

Scientific data refers to any form of data that is collected, observed, measured, or generated as part of scientific research or experimentation. This data is used to support scientific analysis, develop theories, and validate hypotheses. It can come from a wide range of sources, including experiments, simulations, observations, or surveys across various scientific fields.

In general, scientific data can be described ty two terms: types of data and forms of data. They are related but distinct – types describe the nature of the data, while forms describe the how the data is structured and formatted (and stored, which will be discussed below).

Types of scientific data

Types of scientific data refer to what the data represents. It focuses on the nature or category of the data content.

- **Bit and byte**: The smallest unit of storage in a computer is a **bit**, which holds either a 0 or a 1. Typically, eight bits are grouped together to form a **byte**. A single byte (8 bits) can represent up to 256 distinct values. By organizing bytes in various ways, computers can interpret and store different types of data.
- **Numerical data**: Different numerical data types (*e.g.*, integer and floating-point numbers) require different binary representation. Using more bytes for each value increases the range or precision, but it consumes more memory.
    - For example, integers stored with 1 byte (8 bits) have a range from [-128, 127], while with 2 bytes (16 bits) the range becomes [-32768, 32767]. Integers are whole numbers and can be represented exactly given enough bytes.
    - In contrast, floating-point numbers (used for decimals) often suffer from representation errors, since most fractional values cannot be precisely expressed in binary. These errors can accumulate during arithmetic operations. Therefore, in

- For example, integers stored with 1 byte (8 bits) have a range from [-128, 127], while with 2 bytes (16 bits) the range becomes [-32768, 32767]. Integers are whole numbers and can be represented exactly given enough bytes.
- In contrast, floating-point numbers (used for decimals) often suffer from representation errors, since most fractional values cannot be precisely expressed in binary. These errors can accumulate during arithmetic operations. Therefore, in scientific computing, numerical algorithms must be carefully designed to minimize error accumulation. To ensure stability, floating-point numbers are typically allocated 8 bytes (64 bits), keeping approximation errors small enough to avoid unreliable results.
- In ML/DL, half, single, and double precision refer to different formats for representing floating-point numbers, typically using 16, 32, and 64 bits, respectively.
    - **Single precision** (32-bit) is commonly used as a balance between computational efficiency and numerical accuracy.
    - **Half precision** (16-bit) offers faster computation and reduced memory usage, making it popular for training large models on GPUs, though it may suffer from lower numerical stability.
    - **Double precision** (64-bit) provides higher accuracy but is slower and more memory-intensive, so it's mainly used when high numerical precision is critical.
    - Many modern frameworks, like TensorFlow and PyTorch, support mixed precision training, combining half and single precision to optimize performance while maintaining stability.

- **Text data**: When it comes to text data, the simplest character encoding is ASCII (American Standard Code for Information Interchange), which was the most widely used encoding until 2008 when UTF-8 took over. The original ASCII uses only 7 bits for representing each character and therefore can encode 128 specified characters. Later, it became common to use an 8-bit byte to store each character, resulting in extended ASCII with support for up to 256 characters. As computers became more powerful and the need for including more characters from other alphabets, UTF-8 became the most common encoding. UTF-8 uses a minimum of one byte and up to four bytes per character. This flexibility makes UTF-8 ideal for modern applications requiring global character support.
- **Metadata**: Metadata encompasses diverse information about data, including units, timestamps, identifiers, and other descriptive attributes. While most scientific data is either numerical or textual, the associated metadata is usually domain-specific, and different types of data may have different metadata conventions. In scientific applications, such as simulations and experimental results, metadata is typically integrated with the corresponding dataset to ensure proper interpretation and reproducibility.

Forms of scientific data

Forms of scientific data refer to how the data is structured or formatted. It focuses on the presentation or shape of the data.

- **Tabular data structure** (numerical arrays) is a collection of numbers arranged in a specific structure that one can perform mathematical operations on. Examples of numerical arrays are scalar (0D), row or column vector (1D), matrix (2D), and tensor (3D), etc.
- **Textual data structure** is a format for storing and organizing text-based data. It represents unstructured or semi-structured information as sequences of characters (letters, numbers, symbols, punctuation) arranged in strings.
- **Graphs and networks** are forms of scientific data that represent relationships between entities as nodes and connections as edges. They are used to model complex systems
- **Images, videos, and audio** are forms of scientific data that represent information through visual and auditory formats. Images capture static visual information as pixel arrays, videos combine sequential frames to show temporal changes, and audio encodes sound signals as time-series data for analysis.

- **Tabular data structure** (numerical arrays) is a collection of numbers arranged in a specific structure that one can perform mathematical operations on. Examples of numerical arrays are scalar (0D), row or column vector (1D), matrix (2D), and tensor (3D), *etc.*
- **Textual data structure** is a format for storing and organizing text-based data. It represents unstructured or semi-structured information as sequences of characters (letters, numbers, symbols, punctuation) arranged in strings.
- **Graphs and networks** are forms of scientific data that represent relationships between entities as nodes and connections as edges. They are used to model complex systems such as social networks, molecular interactions, and ecological food webs, capturing the structure and connectivity of scientific phenomena.
- **Images, videos, and audio** are forms of scientific data that represent information through visual and auditory formats. Images capture static visual information as pixel arrays, videos combine sequential frames to show temporal changes, and audio encodes sound signals as time-series data for analysis.

Data Storage Format

Representative data storage format

When it comes to data storage, there are many types of storage formats used in scientific computing and data analysis. There isn't one data storage format that works in all cases, so choose a file format that best suits your data.

For tabular data, each column usually has a name and a specific data type while each row is a distinct sample which provides data according to each column (including missing values). The simplest way to save tabular data is using the so-called CSV (comma-separated values) file, which is human-readable and easily shareable. However, it is not the best format to use when working with big (numerical) data.

Gridded data is another very common data type in which numerical data is normally saved in a multi-dimensional grid (array). Common field-agnostic array formats include:

- **Hierarchical Data Format** (HDF5) is a high performance storage format for storing large amounts of data in multiple datasets in a single file. It is especially popular in fields where you need to store big multidimensional arrays such as physical sciences.
- **Network Common Data Form version 4** (NetCDF4) is a data format built on top of HDF5, but exposes a simpler API with a more standardised structure. NetCDF4 is one of the most used formats for storing large data from big simulations in physical sciences.
- **Zarr** is a data storage format designed for efficiently storing large, multi-dimensional arrays in a way that supports scalability, chunking, compression, and cloud-readiness.
- There are more file formats like feather, parquet, xarray and npy to store arrow tables or data frames.

Overview of data storage format

Below is an overview of common data formats (✅ for *good*, 🟨 for *ok/depends on a case*, and ❌ for *bad*) adapted from Aalto university's Python for scientific computing.

Name:	Human readable:	Space efficiency:	Arbitrary data:	Tidy data:	Array data:	Long term storage/sharing:
Pickle	❌	🟨	✅	🟨	🟨	❌
CSV	✅	❌	❌	✅	🟨	✅
Feather	❌	✅	❌	✅	❌	❌
Parquet	❌	🟨	❌	❌	✅	❌
HDF5	❌	✅	❌	❌	✅	✅

Name:	Human readable:	Space efficiency:	Arbitrary data:	Tidy data:	Array data:	Long term storage/sharing:
Pickle	❌	🟨	✅	🟨	🟨	❌
CSV	✅	❌	❌	✅	🟨	✅
Feather	❌	✅	❌	✅	❌	❌
Parquet	❌	🟨	❌	❌	✅	❌
npy	❌	✅	❌	❌	✅	✅
HDF5	❌	✅	❌	❌	✅	✅
NetCDF4	❌	✅	❌	❌	✅	✅
JSON	✅	❌	🟨	❌	❌	✅
Excel	❌	❌	❌	🟨	❌	🟨
Graph formats	🟨	🟨	❌	❌	❌	✅

Data Structures for ML/DL

ML (and DL) models require numerical input, so we must collect adaquate numerical data before training. For ML tasks, multimedia data like image, audio, or video formats should be converted into tabular data or numerical arrays that ML models can process. This conversion enables models to extract meaningful features, such as pixel intensities, audio frequencies or motion patterns, for tasks like classification or prediction.

Numerical array

Numerical array is a collection of numbers arranged in a specific structure that one can perform mathematical operations on. Examples of numerical arrays are scalar (0D), row or column vector (1D), matrix (2D), and tensor (3D), *etc.*

Python offers powerful libraries like NumPy, PyTorch, TensorFlow, and Dask (parallel Numpy) to work with numerical arrays (0D to *n*D).

```
import numpy as np

# 0D (Scalar)
scalar = np.array(5)

# 1D (Vector)
vector = np.array([1, 2, 3])

# 2D (Matrix)
matrix_2D = np.array([[1, 2], [3, 4]])

# 3D (Matrix)
matrix_3D = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
print(matrix_3D.shape)
```

Tensor

In ML and DL, a tensor is a mathematical object used to represent and manipulate multidimensional data. It generalizes scalars, vectors, and matrices to higher dimensions, serving as the fundamental data structure in frameworks like TensorFlow and PyTorch.

Why to use tensors in ML/DL (advantages of Tensor)?

- Generalization of scalars/vectors/matrices: Tensors extend these concepts to any number of dimensions, which is essential for handling complex data like images (3D) and videos (4D+).
- Consistency: Tensors unify data structures across ML/DL frameworks, simplifying model building, training, and deployment.
- Efficient computation: Frameworks like TensorFlow and PyTorch optimize tensor

multidimensional data. It generalizes scalars, vectors, and matrices to higher dimensions, serving as the fundamental data structure in frameworks like TensorFlow and PyTorch.

Why to use tensors in ML/DL (advantages of Tensor)?

- Generalization of scalars/vectors/matrices: Tensors extend these concepts to any number of dimensions, which is essential for handling complex data like images (3D) and videos (4D+).
- Consistency: Tensors unify data structures across ML/DL frameworks, simplifying model building, training, and deployment.
- Efficient computation: Frameworks like TensorFlow and PyTorch optimize tensor operations for speed (using GPUs/TPUs).
- Neural network representations: Input data (images, text) is converted to tensors.
- Automatic differentiation: Tensors support gradient tracking, which is vital for backpropagation in neural networks.

`HERE <>`_ we provide a tutorial about Tensor including

- Tensor creation
- Tensor's properties (*shape*, *dtype*, *ndim*)
- Tensor operations
    - indexing, slicing, transposing
    - element-wise operations: addition, subtraction, *etc.*
    - matrix multiplication(*np.dot*, *torch.matmul*)
    - reshaping, flattening, squeezing, unsqueezing
    - reduction operations: sum, mean, max along axes
    - broadcasting: Rules and examples
- Tensors in DL frameworks
    - moving tensors between CPUs and GPUs (suppose that you can access to GPU cards)

Data Preprocessing

With the huge amount of data at disposal, more and more researchers and industry professionals are finding ways to use this data for research and commercial benefits. However, most of the data available by default is too raw. It is important to preprocess it before it can be used to identify important patterns or can be used to train statistical models that can be used to make predictions.

Data preprocessing refers to steps taken to integrate, clean, transform, and organize raw data into a format that can be effectively used by ML algorithms. It's one of the most critical steps in the ML workflow because high-quality data leads to better model performance.

`HERE <>`_ we provide a tutorial addressing representative steps for preprocessing data in penguins datasets using Python libraries like *numpy*, *pandas*, *matplotlib*, *seaborn*, and *scikit-learn*.

Feature Engineering

Feature engineering is a part of the broader data processing pipeline in ML workflows. It involves using domain knowledge to select, modify, or create new features – variables or attributes – from existing data to help algorithms better understand patterns and relationships.

Feature engineering is crucial because the quality of features directly impacts a model's predictive power. Well-crafted features can simplify complex patterns, reduce overfitting, and improve model interpretability, leading to better generalization and performance on unseen data. By tailoring features to the problem at hand, feature engineering bridges the gap between raw data and actionable insights, often making the difference between a mediocre and a high-performing model.

Feature engineering is a part of the broader data processing pipeline in ML workflows. It involves using domain knowledge to select, modify, or create new features – variables or attributes – from existing data to help algorithms better understand patterns and relationships.

Feature engineering is crucial because the quality of features directly impacts a model's predictive power. Well-crafted features can simplify complex patterns, reduce overfitting, and improve model interpretability, leading to better generalization and performance on unseen data. By tailoring features to the problem at hand, feature engineering bridges the gap between raw data and actionable insights, often making the difference between a mediocre and a high-performing model.

Feature engineering is closely related to data processing, but they serve different purposes.

- Data processing (or data preprocessing) is about cleaning and preparing data – handling missing values, removing duplicates, correcting data types, and ensuring consistency. This step makes the data **usable**.
- Feature engineering, on the other hand, comes after basic processing and focuses on improving the predictive power of dataset.
- In essence, **data processing ensures data quality**, while **feature engineering enhances data value** for ML models.
- Both are essential steps in building effective and accurate predictive systems.

Supervised Learning (I): Classification

❓ Questions

- why

❗ Objectives

- Explain

Instructor note

- 40 min teaching
- 40 min exercises

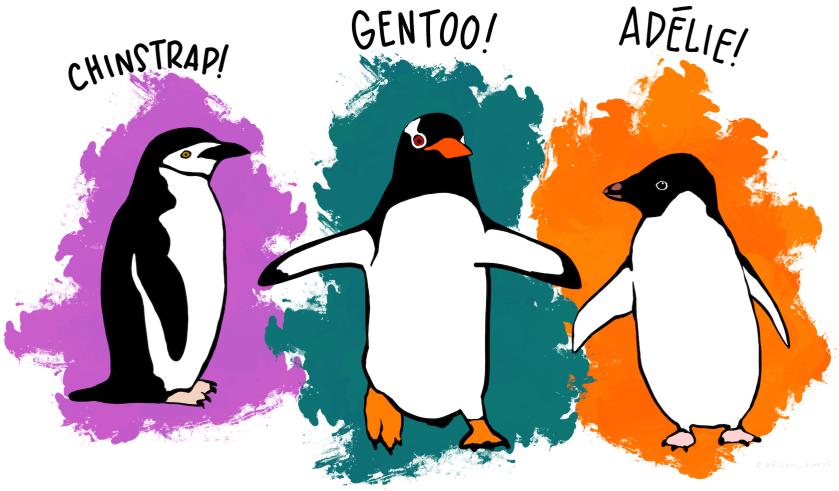
Classification is a supervised ML task where the model predicts discrete class labels based on input features. It involves training a model on labeled data so that it can assign new data to predefined categories or classes based on patterns learned from labeled training data.

In binary classification, models predict one of two classes, such as spam or not spam for emails. Multiclass classification extends this to multiple categories, like classifying images as cats, dogs, or birds.

Common algorithms for classification task include k-Nearest Neighbors (k-NN), logistic Regression, decision tree, random forest, naive Bayes, support vector machine (SVM), gradient boosting, and neural networks.

In this episode we will perform supervised classification tasks to categorize penguins into three species – Adelie, Chinstrap, and Gentoo – based on their physical measurements (flipper length, body mass, *etc.*). We will build and train multiple classifier models as mentioned above. Each model will be evaluated using appropriate performance metrics like accuracy, precision, recall, and F1 score. By comparing the results across models, we aim to identify which classifier model provides the most accurate and reliable classification for this task.

In this episode,we will perform supervised classification tasks to categorize penguins into three species – Adelie, Chinstrap, and Gentoo – based on their physical measurements (flipper length, body mass, *etc.*). We will build and train multiple classifier models as mentioned above. Each model will be evaluated using appropriate performance metrics like accuracy, precision, recall, and F1 score. By comparing the results across models, we aim to identify which classifier model provides the most accurate and reliable classification for this task.



The Palmer Penguins data were collected from 2007-2009 by Dr. Kristen Gorman with the Palmer Station Long Term Ecological Research Program, part of the US Long Term Ecological Research Network. The data were imported directly from the Environmental Data Initiative (EDI) Data Portal, and are available for use by CC0 license ("No Rights Reserved") in accordance with the Palmer Station Data Policy.

Importing Dataset

Seaborn provides the Penguins dataset through its built-in data-loading functions. We can access it using `sns.load_dataset('penguin')` and then have a quick look at the data:

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns

penguins = sns.load_dataset('penguins')
penguins
```

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	sex
0	Adelie	Torgersen	39.1	18.7	181.0	3750.0	Male
1	Adelie	Torgersen	39.5	17.4	186.0	3800.0	Female
2	Adelie	Torgersen	40.3	18.0	195.0	3250.0	Female
3	Adelie	Torgersen	NaN	NaN	NaN	NaN	NaN
4	Adelie	Torgersen	36.7	19.3	193.0	3450.0	Female
...	...	...	...	...	...	...	...
339	Gentoo	Biscoe	NaN	NaN	NaN	NaN	NaN
340	Gentoo	Biscoe	46.8	14.3	215.0	4850.0	Female
341	Gentoo	Biscoe	50.4	15.7	222.0	5750.0	Male
342	Gentoo	Biscoe	45.2	14.8	212.0	5200.0	Female

There are seven columns include:

- *species*: penguin species (Adelie, Chinstrap, Gentoo)
- *island*: island where the penguin was found (Biscoe, Dream, Torgersen)

339	Gentoo	Biscoe	NaN	NaN	NaN	NaN	NaN
340	Gentoo	Biscoe	46.8	14.3	215.0	4850.0	Female
341	Gentoo	Biscoe	50.4	15.7	222.0	5750.0	Male
342	Gentoo	Biscoe	45.2	14.8	212.0	5200.0	Female

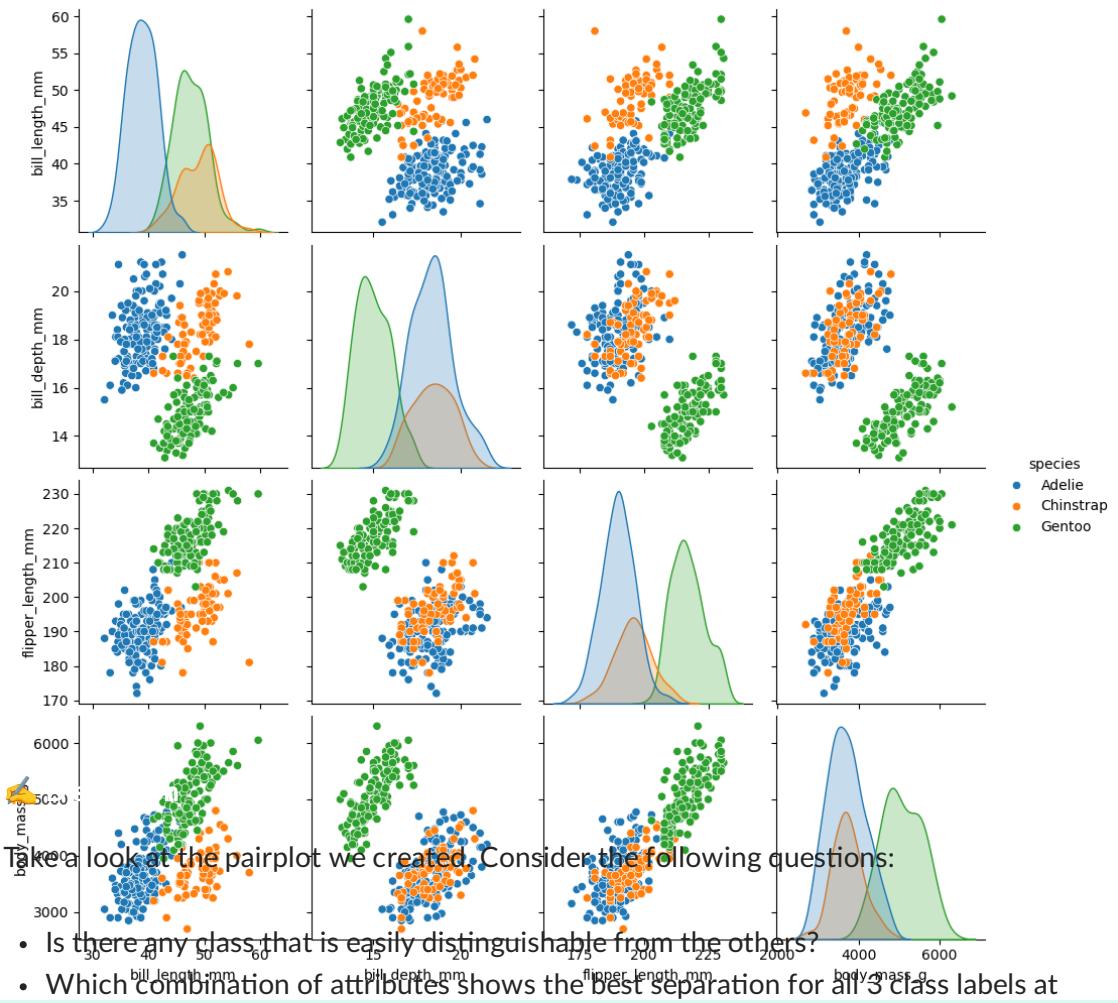
There are seven columns include:

- *species*: penguin species (Adelie, Chinstrap, Gentoo)
- *island*: island where the penguin was found (Biscoe, Dream, Torgersen)
- *bill_length_mm*: length of the bill
- *bill_depth_mm*: depth of the bill
- *flipper_length_mm*: length of the flipper
- *body_mass_g*: body mass in grams
- *sex*: male or female

Looking at numbers from `penguins` and `penguins.describe()` usually does not give a very good intuition about the data we are working with, we have the preference to visualize the data.

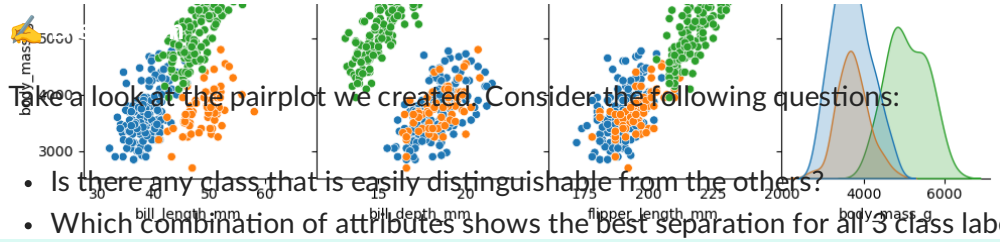
One nice visualization for datasets with relatively few attributes is the Pair Plot, which can be created using `sns.pairplot(...)`. It shows a scatterplot of each attribute plotted against each of the other attributes. By using the `hue='species'` setting for the pairplot the graphs on the diagonal are layered kernel density estimate plots for the different values of the `species` column.

```
sns.pairplot(penguins_classification[["species", "bill_length_mm", "bill_depth_mm",
"flipper_length_mm", "body_mass_g"]], hue="species", height=2.0)
```



Take a look at the pairplot we created. Consider the following questions:

- Is there any class that is easily distinguishable from the others?
- Which combination of attributes shows the best separation for all 3 class labels at once?



✍ Take a look at the pairplot we created. Consider the following questions:

- Is there any class that is easily distinguishable from the others?
- Which combination of attributes shows the best separation for all 3 class labels at once?
- (optional) Create a similar pairplot, but with `hue="sex"`. Explain the patterns you see. Which combination of features distinguishes the two sexes best?

✔ Solution

1. The plots show that the green class (Gentoo) is somewhat more easily distinguishable from the other two.
2. Adelie and Chinstrap seem to be separable by a combination of bill length and bill depth (other combinations are also possible such as bill length and flipper length).
3. `sns.pairplot(penguins_classification, hue="sex", height=2.0)`. From the plots you can see that for each species females have smaller bills and flippers, as well as a smaller body mass. You would need a combination of the species and the numerical features to successfully distinguish males from females. The combination of bill_depth_mm and body_mass_g gives the best separation.

Data Processing

Handling missing values and outliers

For a ML task, the input data (features) and target data (label) are not yet in a right format to use. We need to pre-process the data (as what we did yesterday) to clean missing values using `penguins_classification = penguins.dropna()` and check duplicate values using `penguins_classification.duplicated().value_counts()`.

It is noted that we don't have outliers in this dataset (as we have discussed this issue in the `data processing <>`_ tutorial). For the other datasets you use for the first time, you should check if there are outliers for some features in the dataset, and then take steps to handle the outliers, either to imputate outliers with mean/median values or to remove abnormal outliers for simplicity.

Encoding categorical variables

In the classification task, we will use the categorical variable *species* as the label (target variable), and other columns as features to predict the species of penguins.

✍ Discussion

- why to use *species*?
- why not to use other other categorical variables (here it would be *island* and *sex*)?

✔ Solution

1. *species* will be the main biological classification target in this dataset as it 3 distinct classes (Adelie, Chinstrap, and Gentoo). This is commonly used in ML tutorials as a multi-class classification example (similar to the Iris dataset).
2. *island* is not a ideal label as it is just geographical info, not a biological classification target; *sex* is possible but quite limited. This variable only has two classes (only for binary classification), and the data is unbalanced and has missing values.

- Why not to use other other categorical variables (here it would be *island* and *sex*)?

✔ Solution

1. *species* will be the main biological classification target in this dataset as it 3 distinct classes (Adelie, Chinstrap, and Gentoo). This is commonly used in ML tutorials as a multi-class classification example (similar to the Iris dataset).
2. *island* is not a ideal label as it is just geographical info, not a biological classification target; *sex* is possible but quite limited. This variable only has two classes (only for binary classification), and the data is unbalanced and has missing values.

It is noted that ML models cannot directly process categorical (non-numeric) data, so we have to encode categorical variables like *species*, *island*, and *sex* into numerical values. Here we use `LabelEncoder` from `sklearn.preprocessing` to convert the species column, which serves as our classification target. The `LabelEncoder` assigns a unique integer to each species: "Adelie" becomes 0, "Chinstrap" becomes 1, and "Gentoo" becomes 2. This transformation allows classification algorithms to treat the species labels as distinct, unordered classes.

Then we apply the same rule to encode the island and sex columns. Although these are typically better handled with one-hot encoding due to their nominal nature, we use `LabelEncoder` here for simplicity and compact representation. Each unique category in island (*e.g.*, "Biscoe", "Dream", "Torgersen") and sex (*e.g.*, "Male", "Female") is mapped to a unique integer. This enables us to include them as input features in the model without manual transformation. However, it's important to note that `LabelEncoder` introduces an implicit ordinal relationship, which might not always be appropriate – in such cases, `OneHotEncoder` is preferred.

```
from sklearn.preprocessing import LabelEncoder

encoder = LabelEncoder()

# encode "species" column with 0=Adelie, 1=Chinstrap, and 2=Gentoo
penguins_classification.loc[:, 'species'] =
encoder.fit_transform(penguins_classification['species'])

# encode "island" column with 0=Biscoe, 1=Dream and 2=Torgersen
penguins_classification.loc[:, 'island'] =
encoder.fit_transform(penguins_classification['island'])

# encode "sex" column 0=Female and 1=Male
penguins_classification.loc[:, 'sex'] =
encoder.fit_transform(penguins_classification['sex'])
```

Data Splitting

Splitting features and labels

In preparing the penguins dataset for classification, we first need to split the data into features and labels. The target variable we aim to predict is the penguin species, which we encode into numeric labels using `LabelEncoder`. This encoded species column will be the label vector (e.g., y). The remaining columns – such as bill length, bill depth, flipper length

```
X = penguins_classification.drop(['species'], axis=1)
y = penguins_classification['species'].astype('int')
```

Separating features (X) from labels (y) ensures a clear distinction between what the model uses for prediction and what it is trying to predict.

Splitting training and testing sets

After separating features and labels in the penguins dataset, we further divide the data into a

```
X = penguins_classification.drop(['species'], axis=1)
y = penguins_classification['species'].astype('int')
```

Splitting training and testing sets

Separating features (X) from labels (y) ensures a clear distinction between what the model uses for prediction and what it is trying to predict.
After separating features and labels in the penguins dataset, we further divide the data into a training set and a testing set. The training set is used to train the model, allowing it to learn patterns and relationships from the data, and the test set, on the other hand, is reserved for evaluating the model's performance on unseen data. A common split is 80% for training and 20% for testing, which provides enough data for training while still retaining a meaningful test set.

This splitting is typically done using the `train_test_split` function from `sklearn.model_selection`, with a fixed `random_state` to ensure reproducibility.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=123)

print(f"Number of examples for training is {len(X_train)} and test is {len(X_test)}")
```

Feature scaling

Before training, it is also essential to ensure that numerical features are properly scaled via applying standardization or normalization – especially for distance-based or gradient-based models – to achieve optimal results.

```
from sklearn.preprocessing import StandardScaler

# Standardize features
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Training Model & Evaluating Model Performance

After preparing the Penguins dataset by handling missing values, encoding categorical variables, and splitting it into features-labels and training-test datasets, the next step is to apply classification algorithms including k-Nearest Neighbors (KNN), Decision Trees, Random Forests, Naive Bayes, and Neural Networks to predict penguin species based on their physical measurements Each algorithm offers a unique approach to pattern recognition and generalization, and applying them to the same prepared dataset allows for a fair comparison of their predictive performance.

Below is the generic steps for representative algorithms we will use to training a model for pen

- choosing the model by first importing the class with desired values `xxx_clf = XXXClassifier(<... hyperparameters ...>)`
- training the model to the preprocessed train data by calling the `fit()` method of the model instance `xxx_clf.fit(X_train_scaled, y_train)`
- making predictions using the trained model on test data `y_pred_xxx = xxx_clf.predict(X_test_scaled)`

choosing a mod `sklearn.neighbors import XXXClassifier`

Below is the generic steps for representative algorithms we will use to training a model for pen

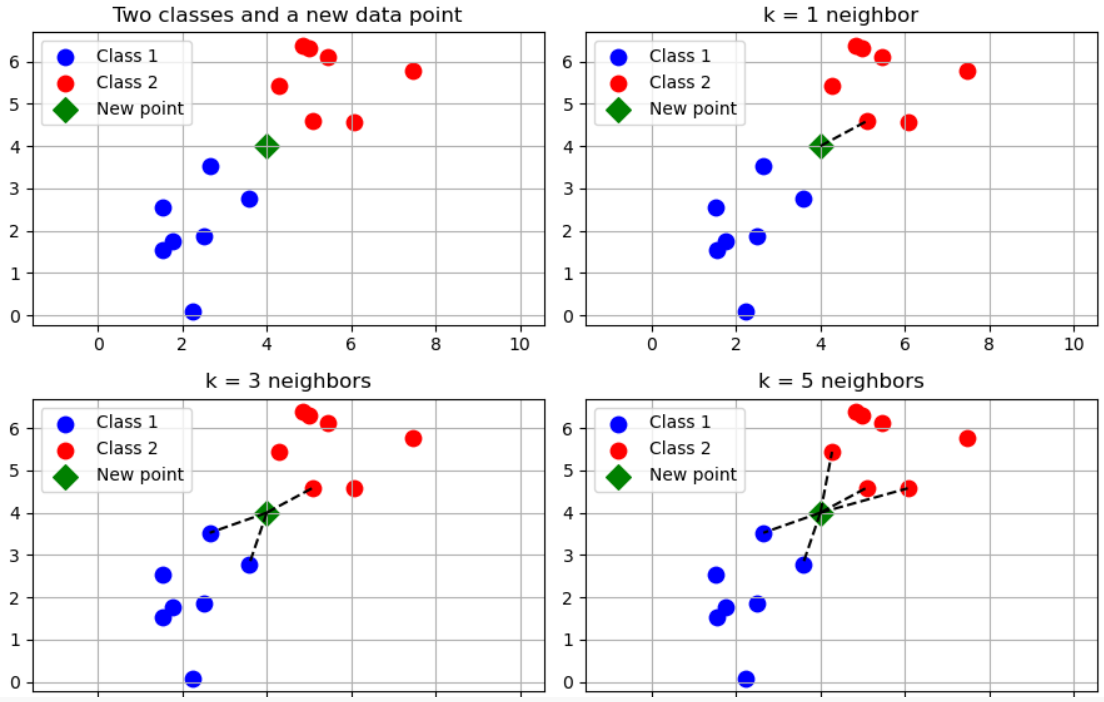
- choosing the model hyper-parameters by instantiating this class with desired values `xxx_clf = XXXClassifier(<... hyperparameters ...>)`
- training the model to the preprocessed train data by calling the `fit()` method of the model instance `xxx_clf.fit(X_train_scaled, y_train)`
- choosing a mod `sklearn.neighbors import XXXClassifier`
- making predictions using the trained model on test data `y_pred_xxx = xxx_clf.predict(X_test_scaled)`
- evaluating model's performance using available metrics `score_xxx = accuracy_score(y_test, y_pred_xxx)`
- (optional) data visualization of confusion matrix and relevant data

k-Nearest Neighbors (KNN)

One intuitive and widely-used method is the k-Nearest Neighbors (KNN) algorithm. KNN is a non-parametric, instance-based algorithm that predicts a sample's label based on the majority class of its *k* closest neighbors in training set.

KNN does not require training in the traditional sense; instead, it stores the entire dataset and performs computation during prediction time. This makes it a lazy learner but potentially expensive during inference.

Here is an example of using the KNN algorithm to determine which class the new point belongs to. When the given query point, the KNN algorithm calculates the distance between this point and all points in the training dataset. It then selects the *k* points that are closest. The class with the most representatives among the *k* neighbors is chosen to be the prediction result for the query point. It is noted that the choice of *k* (the number of neighbors) significantly affects performance: a small *k* may be sensitive to noise, while a large *k* may smooth over important patterns.




```
from sklearn.neighbors import KNeighborsClassifier

knn_clf = KNeighborsClassifier(n_neighbors=3)
knn_clf.fit(X_train_scaled, y_train)
```

After we fitting the training data, we use the trained model to predict species on the test set

```
from sklearn.neighbors import KNeighborsClassifier

knn_clf = KNeighborsClassifier(n_neighbors=3)
knn_clf.fit(X_train_scaled, y_train)
```

After we fitting the training data, we use the trained model to predict species on the test set and evaluate its performance.

For classification tasks, metrics like accuracy, precision, recall, and the F1-score provide a comprehensive view of model performance.

- **accuracy** measures the proportion of correctly classified instances across all species (Adelie, Chinstrap, Gentoo), and it gives an overall measure of how often the model is correct, but it can be misleading for imbalanced datasets.
- **precision** quantifies the proportion of correct positive predictions for each species, while **recall** assesses the proportion of actual positives correctly identified.
- the **F1-score**, the harmonic mean of precision and recall, balances these metrics for each class, especially useful given the dataset's imbalanced species distribution.

```
# predict on test data
y_pred_knn = knn_clf.predict(X_test_scaled)

# evaluate model performance
from sklearn.metrics import classification_report, accuracy_score

score_knn = accuracy_score(y_test, y_pred_knn)

print("Accuracy for k-Nearest Neighbors:", score_knn)
print("\nClassification Report:\n", classification_report(y_test, y_pred_knn))
```

In classification tasks, a **confusion matrix** is a valuable tool for evaluating model performance by comparing predicted labels against true labels. For a multiclass classification task like the penguins dataset, the confusion matrix is an **N x N** matrix, where **N** is the number of target classes (here **N=3** for three penguins species). Each cell *(i, j)* in the matrix indicates the number of instances where the true class was *i* and the model predicted class *j*. Diagonal elements represent correct predictions, while off-diagonal elements indicate misclassifications. The confusion matrix provides an easy-to-understand overview of how often the predictions match the actual labels and where the model tends to make mistakes.

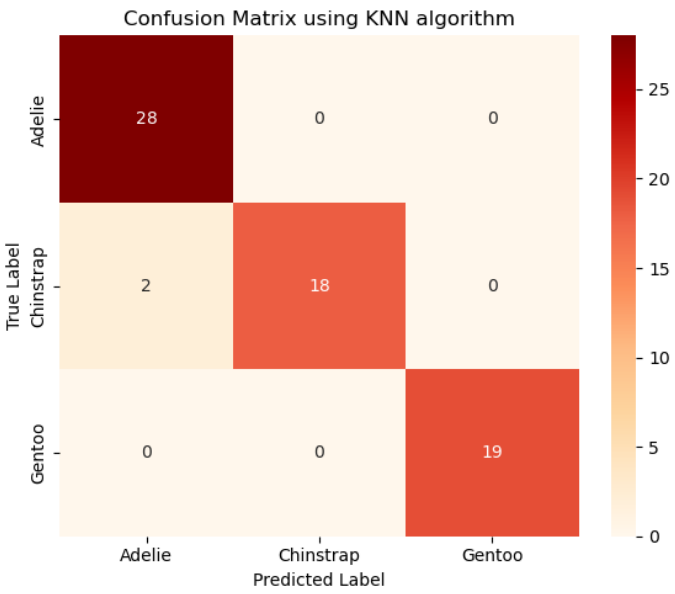
Since we will plot the confusion matrix multiple times, we write a function and call this function later whenever needed, which promotes clarity and avoids redundancy. This is especially helpful as we evaluate multiple classifiers such as KNN, Decision Trees, or SVM on the penguins dataset.

```
from sklearn.metrics import confusion_matrix

def plot_confusion_matrix(conf_matrix, title, fig_name):
    plt.figure(figsize=(6, 5))
    sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='OrRd',
                xticklabels=["Adelie", "Chinstrap", "Gentoo"],
                yticklabels=['Adelie', 'Chinstrap', 'Gentoo'], cbar=True)
```

```
from sklearn.metrics import confusion_matrix

def plot_confusion_matrix(conf_matrix, title, fig_name):
    plt.figure(figsize=(6, 5))
    sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='OrRd',
                xticklabels=["Adelie", "Chinstrap", "Gentoo"],
                yticklabels=['Adelie', 'Chinstrap', 'Gentoo'], cbar=True)

    plt.xlabel("Predicted Label")
    plt.ylabel("True Label")
    plt.title(title)
    plt.tight_layout()
    plt.savefig(fig_name)
```

We compute the confusion matrix from the trined model using the KNN algorithm, and visualize the matrix.

```
cm_knn = confusion_matrix(y_test, y_pred_knn)

plot_confusion_matrix(cm_knn, "Confusion Matrix using KNN algorithm", "confusion-matrix-knn.png")
```



The first row: there are 28 Adelie penguins in the test data, and all these penguins are identified as Adelie (valid). The second row: there are 20 Chinstrap pengunis in the test data, with 2 identified as Adelie (invalid), none are correctly recognized as Chinstrap, and 18 identified as Chinstrap (valid). The third row: there are 19 Gentoo penguins in the test data, and all these penguins are identified as Gentoo (valid).

Logistic Regression

Logistic Regression is a fundamental classification algorithm to predict categorical outcomes. Despite its name, logistic regression is not a regression algorithm but a classification method that predicts the probability of an instance belonging to a particular class.

For a multiclass classification, logistic regression can be extended using strategies like **one-vs-rest** (OVR) or softmax regression.

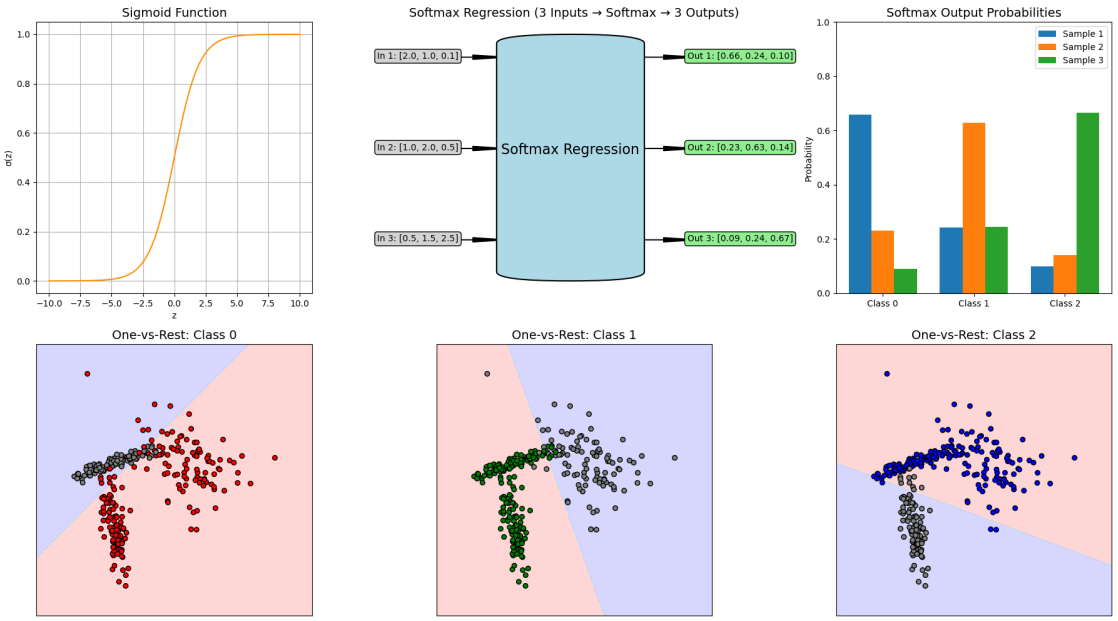
For binary classification, it uses the logistic (**sigmoid**) function to map a linear combination of input features to a probability between 0 and 1, which is then thresholded (typically at 0.5) to assign a class.

- In OVR, a separate binary classifier is trained for each species against all others.
- **softmax regression** generalizes the logistic function to compute probabilities across all classes simultaneously, selecting the class with the highest probability.

For a multiclass classification, logistic regression can be extended using strategies like **one-vs-rest** (OVR) or softmax regression.

For binary classification, it uses the logistic (**sigmoid**) function to map a linear combination of input features to a probability between 0 and 1, which is then thresholded (typically at 0.5) to assign a class.

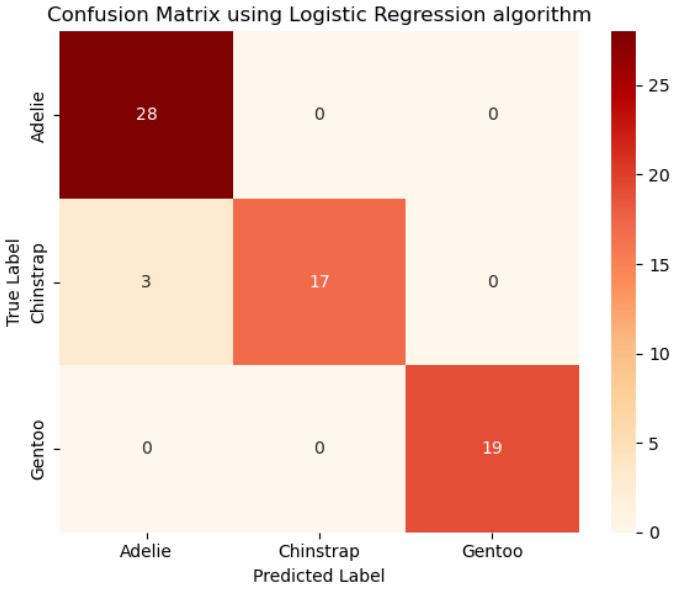
- In OVR, a separate binary classifier is trained for each species against all others.
- **softmax regression** generalizes the logistic function to compute probabilities across all classes simultaneously, selecting the class with the highest probability.



(Upper left) the sigmoid function; (upper middle) the softmax regression process: three input features to the softmax regression model resulting in three output vectors where each contains the predicted probabilities for three possible classes; (upper right) a bar chart of softmax outputs in which each group of bars represents the predicted probability distribution over three classes; lower subplots) three binary classifiers distinguish one class from the other two classes using the one-vs-rest approach.

The creation of a Logistic Regression model and the process of fitting it to the training data are nearly identical to those used for the KNN model described above, except that a different classifier is selected. The code example and the resulting confusion matrix plot are provided below:

```
from sklearn.linear_model import LogisticRegression

lr_clf = LogisticRegression(random_state = 0)
lr_clf.fit(X_train_scaled, y_train)

y_pred_lr = lr_clf.predict(X_test_scaled)

score_lr = accuracy_score(y_test, y_pred_lr)
print("Accuracy for Logistic Regression:", score_lr )
print("\nClassification Report:\n", classification_report(y_test, y_pred_lr))

cm_lr = confusion_matrix(y_test, y_pred_lr)
plot_confusion_matrix(cm_lr, "Confusion Matrix using Logistic Regression algorithm",
"confusion-matrix-lr.png")
```



Naive Bayes

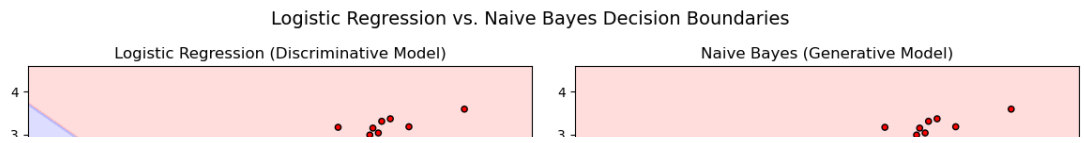
The **Naive Bayes** algorithm is a simple yet powerful probabilistic classifier based on Bayes' Theorem. This classifier assumes that all features are equally important and independent which is often not the case and may result in some bias. However, the assumption of independence simplifies the computations by turning conditional probabilities into products of probabilities. This algorithm computes the probability of each class given the input features and selects the class with the highest posterior probability.

Logistic regression and Naive Bayes are both popular algorithms for classification tasks, but they differ significantly in their approach, assumptions, and mechanics.

- Logistic regression is a **discriminative** model that directly models the probability of a data point belonging to a particular class by fitting a linear combination of features through a logistic (sigmoid) function for binary classification or softmax for multiclass tasks. For the penguins dataset, it would use features like bill length and flipper length to compute a weighted sum, transforming it into probabilities for species like Adelie, Chinstrap, or Gentoo. It assumes a linear relationship between features and the log-odds of the classes and optimizes parameters using maximum likelihood estimation, making it sensitive to feature scaling and correlations. Logistic regression is robust to noise and can handle correlated features to some extent, but it may struggle with highly non-linear relationships unless feature engineering is applied.
- Naive Bayes, in contrast, is a **generative** model that relies on Bayes' theorem to compute the probability of a class given the features, assuming conditional independence between features given the class. For the penguins dataset, it would estimate the likelihood of features (*e.g.*, bill depth) for each species and combine these with prior probabilities to predict the most likely species. The "naive" assumption of feature independence often doesn't hold (*e.g.*, bill length and depth may be correlated), but Naive Bayes is computationally efficient, works well with high-dimensional data, and is less sensitive to irrelevant features. However, it can underperform when feature dependencies are significant or when the data distribution deviates from its assumptions (*e.g.*, Gaussian for continuous features in Gaussian Naive Bayes). Unlike logistic regression, it doesn't require feature scaling but may need careful handling of zero probabilities (*e.g.*, via smoothing).

linear decision boundary directly, while Naive Bayes models feature distributions per class (assuming independence).

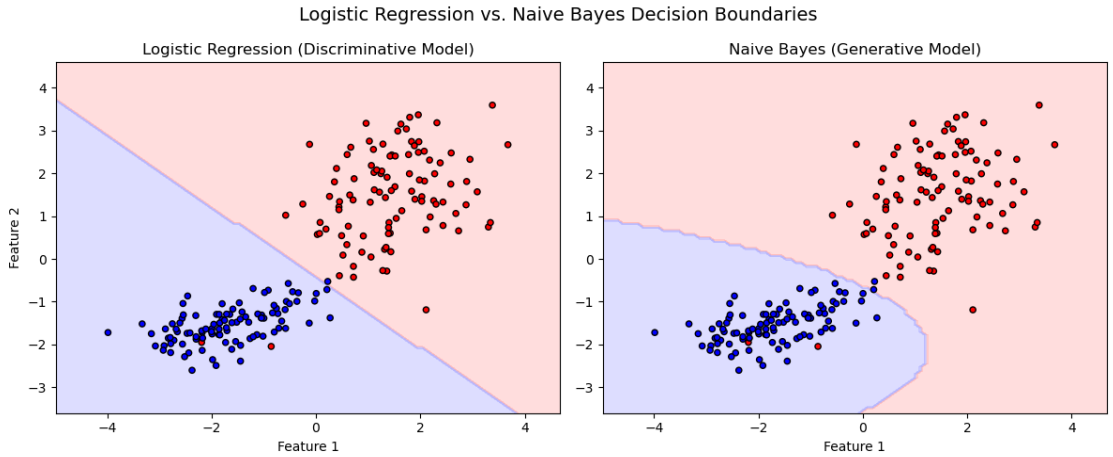
E a
s
d



linear decision boundary directly, while Naive Bayes models feature distributions per class (assuming independence).

continuous features in Gaussian Naive Bayes). Unlike logistic regression, it doesn't require feature scaling but may need careful handling of zero probabilities (*e.g.*, via smoothing).

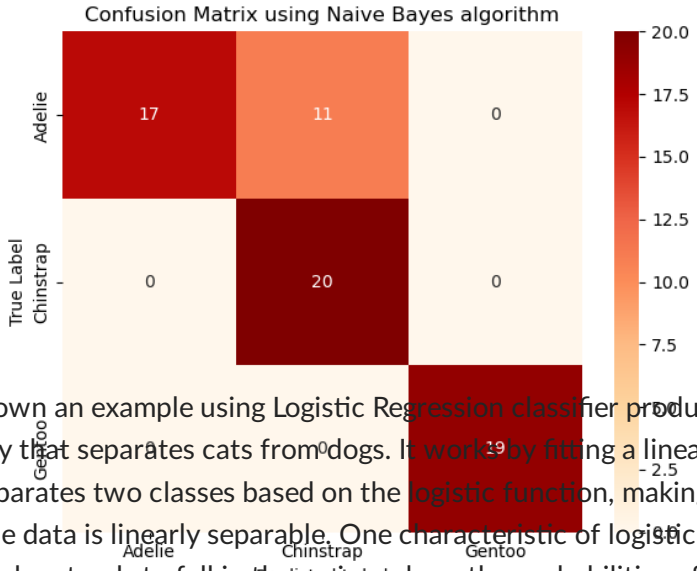
E a s d



To apply Naive Bayes, we use `GaussianNB` from `sklearn.naive_bayes`, which assumes that the features follow a Gaussian (normal) distribution, which is an appropriate choice for continuous numerical data such as bill length and body mass. Since Naive Bayes relies on probabilities, **feature scaling is not required**, but **handling missing values and encoding categorical variables numerically is still necessary**.

While Naive Bayes may not outperform more complex models like Random Forests, it offers **fast training, low memory usage**, and good performance for simple tasks.

```
from sklearn.naive_bayes import GaussianNB

nb_clf = GaussianNB()
nb_clf.fit(X_train_scaled, y_train)

y_pred_nb = nb_clf.predict(X_test_scaled)

score_nb = accuracy_score(y_test, y_pred_nb)
print("Accuracy for Naive Bayes:", score_nb)
print("\nClassification Report:\n", classification_report(y_test, y_pred_nb))

cm_nb = confusion_matrix(y_test, y_pred_nb)
plot_confusion_matrix(cm_nb, "Confusion Matrix using Naive Bayes algorithm",
"confusion-matrix-nb.png")
```



Previously we shown an example using Logistic Regression classifier producing a linear decision boundary that separates cats from dogs. It works by fitting a linear decision boundary that separates two classes based on the logistic function, making it particularly effective when the data is linearly separable. One characteristic of logistic regression is that the decision boundary tends to fall in the region where the probabilities of two classes are closest—typically where the model is most uncertain.

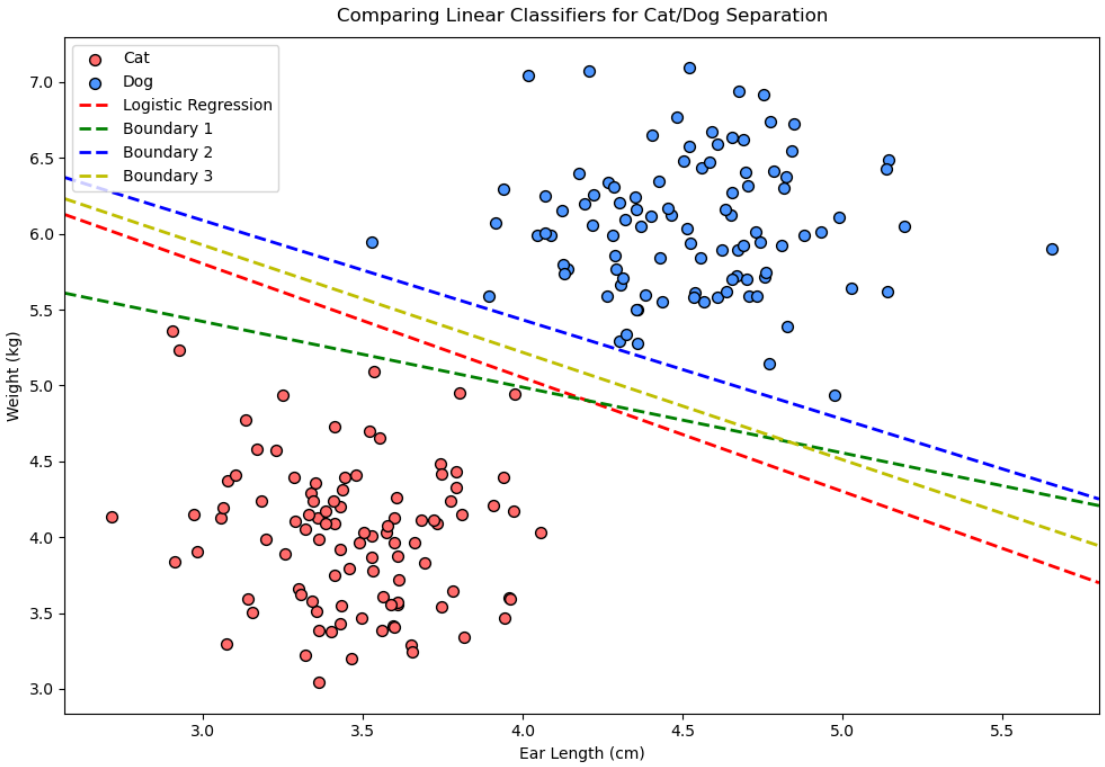
Support Vector Machine (SVM)

Previously we shown an example using Logistic Regression classifier producing a linear decision boundary that separates cats from dogs. It works by fitting a linear decision boundary that separates two classes based on the logistic function, making it particularly effective when the data is linearly separable. One characteristic of logistic regression is that the decision boundary tends to fall in the region where the probabilities of two classes are closest – typically where the model is most uncertain.

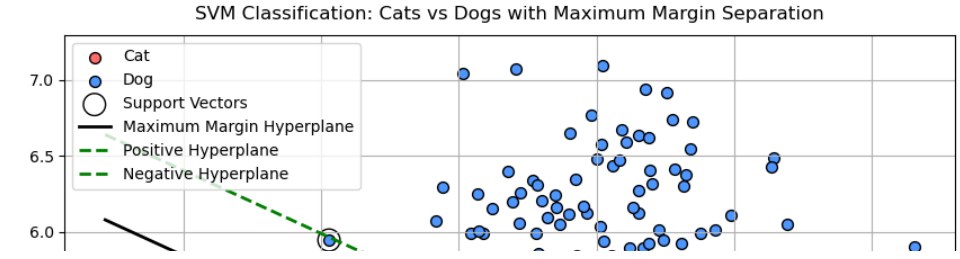
Support Vector Machine (SVM)

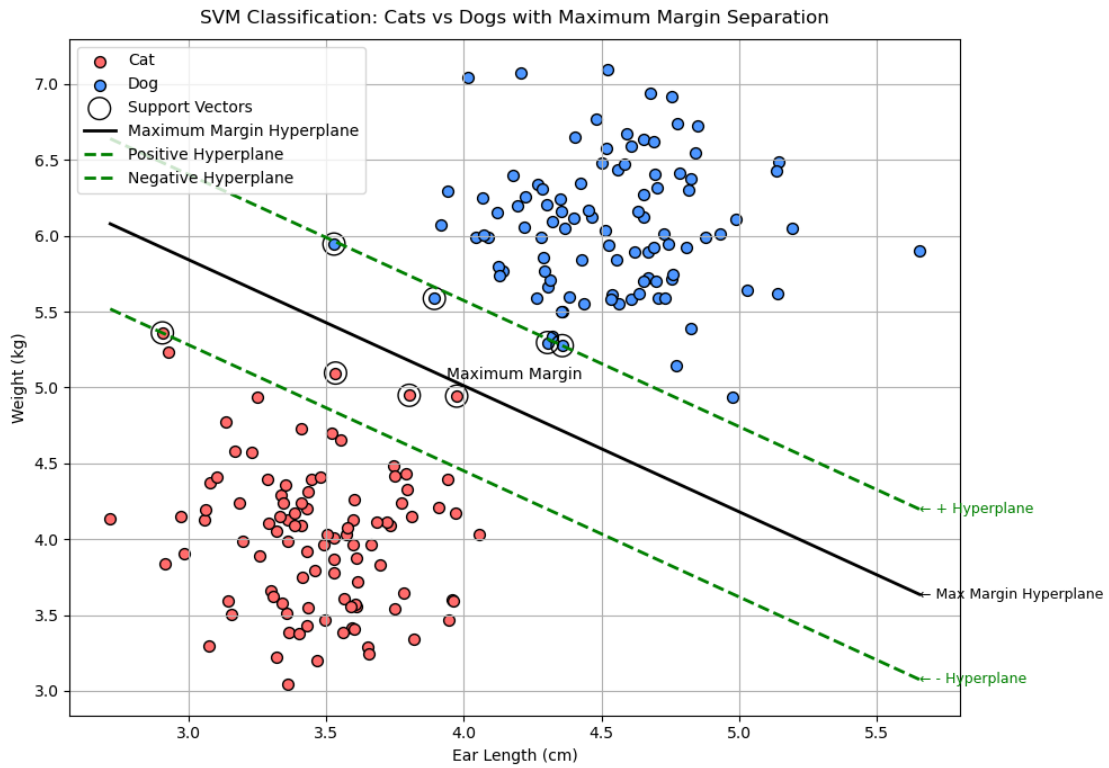
However, when there exists a large gap between two well-separated classes – as often occurs when distinguishing cats and dogs based on weight and ear length – logistic regression faces an inherent limitation: infinite possible solutions. The algorithm has no mechanism to select an "optimal" boundary when multiple valid linear separators exist in the wide margin between classes, and it will place the decision boundary somewhere in that gap, leading to a broad, undefined decision region with no supporting data. While this may not affect accuracy on clearly separated data, it can make the model less robust when new or noisy data appears near that boundary.

Below is an example, again, to separate cats from dogs based on ear length and weight. Besides the linear decision boundary from Logistic Regression classifier, we can find three additional linear boundaries that can also have a good separation of cats from dogs. Which one is better than the others and how to evaluate their performance on unseen data?



To better handle such situation, we can transition to the **Support Vector Machine** (SVM) algorithm. SVM takes a different approach by focusing on the concept of maximizing the margin – the distance between the decision boundary and the closest data points from each class (the support vectors) (as shown in the figure below). When there is a large gap between the two classes, SVM utilizes that space effectively by pushing the boundary toward the center of the gap while maintaining the maximum margin. This leads to a more stable and





The SVM classification boundary for distinguishing cats and dogs based on ear length and weight. The solid black line represents the maximum margin hyperplane (decision boundary), while the dashed green lines show the positive and negative hyperplanes that define the margin. Black circles highlight the support vectors - the critical data points that determine the margin width.

To apply SVM, we use `SVC` (Support Vector Classification) from `sklearn.svm`, which by default assumes that the features follow a nonlinear relationship modeled by the `rbf` (Radial Basis Function) kernel. This kernel allows the model to find complex decision boundaries by implicitly mapping the input features into a higher-dimensional space. You can easily change the kernel to `linear`, `poly`, or `sigmoid` to experiment with different decision boundaries.

By adjusting the hyperparameters such as `C` (regularization strength) and `gamma` (kernel coefficient), we can control the trade-off between the margin width and classification accuracy. Below is a code example demonstrating how to use SVC with the RBF kernel for the penguins classification task.

```
from sklearn.svm import SVC

svm_clf = SVC(kernel='rbf', C=1.0, gamma='scale', random_state=0)
svm_clf.fit(X_train_scaled, y_train)

y_pred_svm = svm_clf.predict(X_test_scaled)

score_svm = accuracy_score(y_test, y_pred_svm)
print("Accuracy for Support Vector Machine:", score_svm)
print("\nClassification Report:\n", classification_report(y_test, y_pred_svm))

cm_svm = confusion_matrix(y_test, y_pred_svm)
plot_confusion_matrix(cm_svm, "Confusion Matrix using Support Vector Machine
algorithm", "confusion-matrix-svm.png")
```

Decision Tree

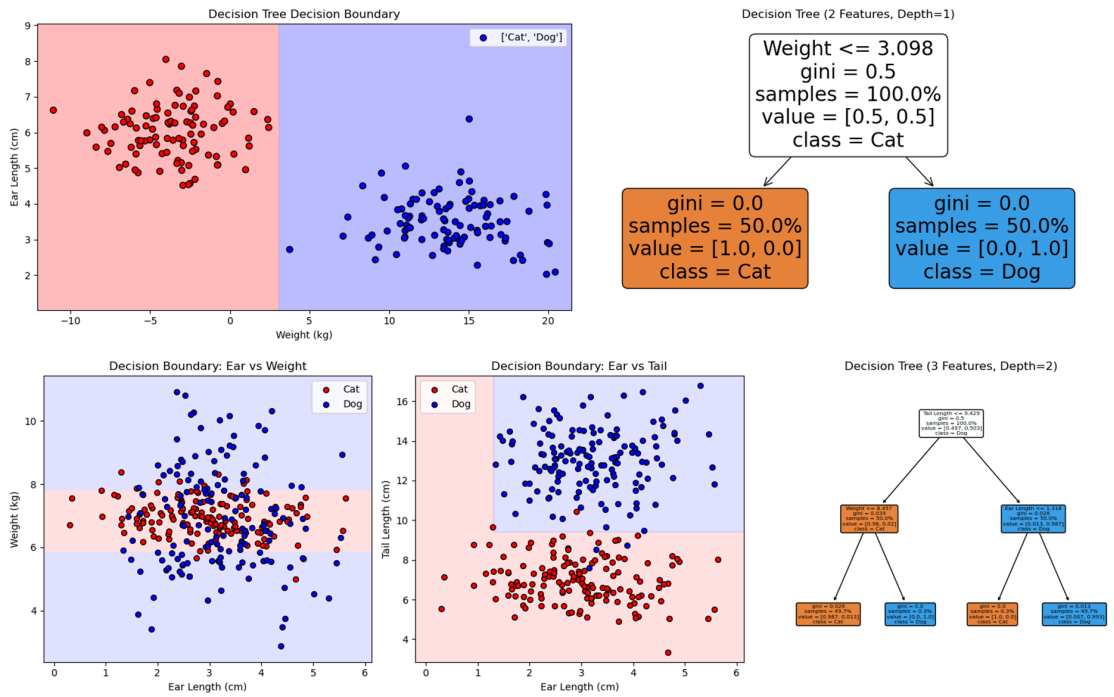
Decision Tree algorithm is a versatile and interpretable method for classification tasks. The core idea of this algorithm is to recursively split the dataset into smaller subsets based on feature thresholds creating a tree-like structure of decisions that result in the most significant separation of target classes.

Here is one example showing how to separate cats and dogs on the basis of two or three

Decision Tree

Decision Tree algorithm is a versatile and interpretable method for classification tasks. The core idea of this algorithm is to recursively split the dataset into smaller subsets based on feature thresholds creating a tree-like structure of decisions that result in the most significant separation of target classes.

Here is one example showing how to separate cats and dogs on the basis of two or three features.



(Upper) decision boundary separating cats and dogs based on two features (ear length and weight), and the corresponding decision tree structure; (lower): two decision boundaries separating cats and dogs based on three features (ear length, weight, and tail length), and the corresponding decision tree structure.

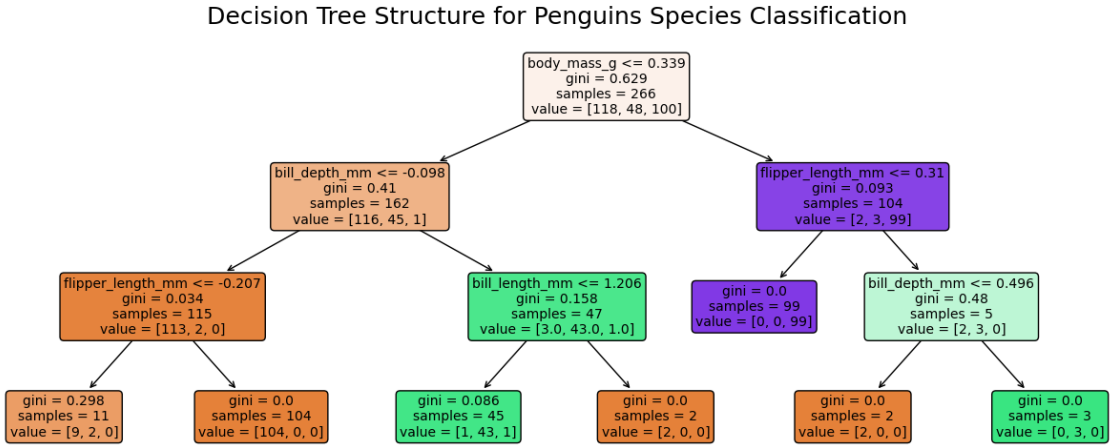
The code example for the Decision Tree classifier is provided below.

```
from sklearn.tree import DecisionTreeClassifier

dt_clf = DecisionTreeClassifier(max_depth=3, random_state = 0)
dt_clf.fit(X_train_scaled, y_train)

y_pred_dt = dt_clf.predict(X_test_scaled)

score_dt = accuracy_score(y_test, y_pred_dt)
print("Accuracy for Decision Tree:", score_dt )
print("\nClassification Report:\n", classification_report(y_test, y_pred_dt))

cm_dt = confusion_matrix(y_test, y_pred_dt)
plot_confusion_matrix(cm_dt, "Confusion Matrix using Decision Tree algorithm",
"confusion-matrix-dt.png")
```

We visualize the Decision Tree structure to understand how penguins are classified based on

```
from sklearn.tree import plot_tree

plt.figure(figsize=(16, 6))
plot_tree(dt_clf, feature_names=X.columns, filled=True, rounded=True, fontsize=10)

plt.title("Decision Tree Structure for Penguins Species Classification", fontsize=18)
```

```
from sklearn.tree import plot_tree

plt.figure(figsize=(16, 6))
plot_tree(dt_clf, feature_names=X.columns, filled=True, rounded=True, fontsize=10)

plt.title("Decision Tree Structure for Penguins Species Classification", fontsize=18)
```



Quick Reference

Instructor's guide

Why we teach this lesson

Intended learning outcomes

Timing

Preparing exercises

e.g. what to do the day before to set up common repositories.

Other practical aspects

Interesting questions you might get

Typical pitfalls

Who is the course for?

About the course

See also

Credits